Service-Level Extraction: ADK Mode and Pipeline Mode

A step-by-step guide: from installing the tool to extracting, reviewing and saving the services in a waiver

> **IMPORTANT: THIS TOOL NEEDS GOOGLE CLOUD CREDITS OR A BILLING ACCOUNT.**
> **Every extraction sends the waiver PDF to a Gemini model on your Google Cloud project, and that usage is charged to the project's billing account. Services extraction makes many model calls per waiver (at least one per service). Make sure billing is enabled, or that you have free credits, before your first run.** See section 2.2.

Covers: the service-level app (`services_extraction_agent`), used through the ADK web interface (ADK mode) and from the command line (pipeline mode). Waiver-level (overview) extraction has its own guide.
Tested with: macOS, Python 3.12, Google ADK 2.0.0, Gemini 3.7 Flash, Google Chrome. **Not tested on Windows.**
Guide written: October 20, 2026
Updated: November 1, 2026

Table of contents

1. What this tool does **4**

1.1 Parent services and child services 4

1.2 What a service row contains 4

1.3 The two ways to use the tool (both covered here) 5

1.4 How the tool checks its own work 5

1.5 How accurate is it? 5

1.6 What this guide covers, and what it does not 5

2. Before you start **6**

2.1 What you need 6

2.2 Credits and billing 6

3. Install and set up **7**

3.1 Get the project folder 7

3.2 Create a private Python environment 7

3.3 Install the Python packages 7

3.4 Install poppler 8

3.5 Install the Google Cloud command-line tool and sign in 8

3.6 Find your Google Cloud project ID, enable billing and turn on Vertex AI 8

3.7 Create your .env file 8

4. Start the app and open it in your browser **9**

4.1 Start it 9

4.2 Open the page and choose the services app 9

4.3 Stop the app 10

5. A tour of the screen **10**

6. Extract the services in a waiver **11**

6.1 Tell the tool which waiver 11

6.2 What happens during a full extraction 11

6.3 Timing and cost 12

6.4 The result summary 12

6.5 Resuming an interrupted run 13

6.6 Extract or re-extract only named services 13

6.7 Queue (batch) mode 13

7. Review and correct the results **14**

7.1 How the numbering works 14

7.2 The list of parent services 14

7.3 What the codes mean 14

7.4 Opening a service 15

7.5 Searching 15

7.6 Commands you can type 15

7.7 Changing a value 16
7.8 Rejecting a service or a child 16
7.9 Re-extracting a service with a hint 17
7.10 Writing the CSV 17

8. Extract a value about a service 17
8.1 Ask for it 17
8.2 Confirm what you are asking for 17
8.3 The result 18
8.4 Your four choices 18

9. Pipeline mode (command line) 19
9.1 Before you run a command 19
9.2 Where the list of services comes from 19
9.3 Commands 19
9.4 Reading a command in plain English 20
9.5 Results 20

10. Where your files are saved 21

11. Good habits 22

12. Troubleshooting 22

13. Quick reference 23
13.1 Setup commands (once) 23
13.2 Each time you work 23
13.3 Chat commands 23
13.4 Pipeline commands 24

Appendix A. The .env settings 24

Appendix B. Google documentation and credits 25

Appendix C. Glossary 25

Appendix D. Tested with 26

Appendix E. Setting up with an AI assistant 26
E.1 Who does what 27
E.2 The prompt 27
E.3 What a successful setup looks like 28

1. What this tool does

A 1915(c) Home and Community-Based Services (HCBS) waiver application lists the services the waiver pays for (for example Day Habilitation, Respite or Personal Care), with details for each one: who may provide it, how it is delivered, and for each of up to five years how many people use it, how many units each person uses and what it costs. This tool reads one waiver PDF and turns those details into **one row per service**, in the format of the project's service-level results sheet.

For every value it also records **where it found the answer**: the PDF page and the exact wording. You review the result and decide what is kept. In ADK mode the results CSV is written when you approve, change or reject (section 7.6).

1.1 Parent services and child services

Services form a two-level tree, and the whole tool works with that structure.

- A **parent service** is a service as the waiver names it, for example **Day Habilitation**.
- A **child service** is a variant of a parent that has its own users and costs, for example **Individual Day Habilitation** and **Group Day Habilitation**, both children of Day Habilitation. A parent can have many children (in the project's collection, one parent has 37), or none.

Each parent and each child becomes its own row. In the review step, a parent is numbered **1** and its children are **1.1, 1.2, ...**, so you can always say exactly which row you mean.

1.2 What a service row contains

Group	Fields (internal names in brackets)
Identity	Service name, parent service (for a child), HCBS taxonomy (`hcbs_taxonomy`)
Service details (parents only)	Service type 1 to 4 (`service_type`), delivery method 1 to 3 (`service_delivery_method`), provider eligibility (`provider_eligibility`), provider specifications (`provider_specifications`), self-direction authority (`self_direction_authority`), information and assistance (`information_assistance`)
For each year 1 to 5	Capitation (`year_N_capitation`), unit (`year_N_unit`), users (`year_N_users`), average units per user (`year_N_avg_units_per_user`), average cost per unit (`year_N_avg_cost_per_unit`), component cost (`year_N_component_cost`), total cost (`year_N_total_cost`)

The results CSV has 49 sheet columns plus `Row Type`, `Parent Service` and `Waiver Key`. Section 7.3 explains the codes used in the service details.

1.3 The two ways to use the tool (both covered here)

Mode	What it is	Best for
ADK mode (sections 4 to 8)	A chat window in your web browser. You type plain-English requests and the tool answers, one waiver at a time, with a review step before anything is saved to the CSV.	Extracting and reviewing a waiver, fixing individual services or fields, extracting an extra value about a service.
Pipeline mode (section 9)	Commands typed in a terminal that process one waiver or many in a row, with no chat and no review step.	Large batches, where you check the results afterwards.

1.4 How the tool checks its own work

After the model reads each service, the tool checks the result before showing it to you:

- **Form data.** Many waiver PDFs contain fillable-form data (called AcroForm data). The tool compares what the model read with that data and corrects conflicts.
- **Consistency checks.** For example, a parent's yearly total cost should equal the sum of its children's component costs, and users times average units times average cost should equal the cost shown. Anything that does not add up is **flagged**, and the flags appear in the review table.
- **A second reading.** If flagged problems remain, the tool re-reads those services once more and checks again. There is no third attempt.
- **An accuracy score.** The checker reports an accuracy for each waiver. A run passes at 85% or above, or when nothing is left flagged.

> **Check the results**
> In ADK mode you review every service, with its evidence and flags, before it is written to the CSV. In pipeline mode there is no review step, so check the results before you use them.

1.5 How accurate is it?

Service-level extraction was checked by a person together with Claude. **We are 95% confident that 96% of the extracted service-level values are correct.**

Some values will still be wrong, which is why every value comes with its page and evidence, and why ADK mode has a review step.

1.6 What this guide covers, and what it does not

- **Covered:** setting up, starting the app, extracting every service in a waiver or only named services, queue (batch) mode, resuming an interrupted run, reviewing and correcting the results, writing the CSV, extracting a custom value about a service, and running the same extraction from the command line.
- **Not covered:** waiver-level (overview) extraction. That is a different app, `waiver_level_extraction_agent`, with its own guide.

2. Before you start

2.1 What you need

Item	Details
A computer	macOS was used for everything in this guide. Linux commands are given but were not run. **This project has not been tested on Windows** (see the warning below).
Python 3.10 or newer	Check with `python3 --version`. This guide was tested with Python 3.12.
A personal Google account	Your own account, with your own Google Cloud project (see the note on institutional accounts below).
A Google Cloud project with billing	See 2.2 below. You will need the project's **ID**.
Google Chrome (recommended)	The chat window runs in a web browser. Chrome was used for every screenshot here.
The project folder	The code, plus the waiver PDFs you want to process.
poppler	A small system tool used to read PDF page images. Installed in section 3.4.

Not tested on a Windows laptop
Everything in this guide was run and checked on macOS. The commands and behaviour on Windows have **not** been tested, so some steps may fail or need changes. Windows alternatives are shown in places but are untested. If you use Windows, consider Windows Subsystem for Linux (WSL), and expect to troubleshoot.

Work, school or institutional Google accounts
If you sign in with an account from an organization, the organization may restrict what you can do. You may need an administrator to give you access to a project, enable billing, or enable specific APIs (such as Vertex AI). **Those steps are not covered in this guide.** The guide assumes you run the extraction in your **own** Google account and your own Google Cloud project.

2.2 Credits and billing

THESE TOOLS NEED CREDITS. EXTRACTION IS NOT FREE.
Each extraction makes many calls to Google's Gemini models through Vertex AI, and Google charges for them. The charges go to the billing account attached to your Google Cloud project. If billing is not enabled, or your credits have run out, extraction will fail.

- According to Google's documentation, new Google Cloud customers can receive free trial credit (**$300, valid for 90 days** at the time of writing). The trial ends when the credit is used up or 90 days pass, whichever is first. Check Google's page for the current terms.
- Vertex AI setup requires billing to be enabled on the project.
- Prices change. **Check the live pricing page before large runs:** `cloud.google.com/vertex-ai/pricing`. This guide does not quote dollar prices because they would go out of date.

- How much one waiver costs depends on its length and on how many fields need a second look. Services extraction makes **one model call per service**, plus calls to find the service list, check the results and re-read anything flagged, so a waiver with many services makes many calls. In our test, extracting a single service from a waiver and running it through the rest of the pipeline took about **4 minutes**; one custom value took about **30 seconds** and one call.
- Watch your spend in the Google Cloud console under **Billing**, and consider setting a budget alert there.

Google's own instructions for this: *Set up a project and a development environment* (docs.cloud.google.com/vertex-ai/docs/start/cloud-environment) and the *Billing questions* page for Vertex AI. See Appendix B for links.

3. Install and set up

You only do this section once per computer. Commands are shown for macOS and Linux. Type them in the **Terminal** app. Lines starting with `#` are comments and are not typed.

> **Prefer to have an AI assistant help?**
> If you already work with an AI assistant, Appendix E has a ready-to-paste prompt that walks it through this setup safely.

3.1 Get the project folder

```
# Download the project (replace the address with your repository's address)
git clone <repository-url> 1915c_extraction_agent
cd 1915c_extraction_agent
```

All later commands are run from inside this folder, called the **project root**. It contains `run_pipeline.py`, `waiver_level_extraction_agent/`, `services_extraction_agent/` and `data/`.

3.2 Create a private Python environment

```
python3 -m venv .venv
source .venv/bin/activate          # macOS / Linux
# Windows PowerShell:  .venv\Scripts\Activate.ps1
```

When the environment is active, your terminal prompt starts with `(.venv)`. You must activate it again each time you open a new terminal.

3.3 Install the Python packages

```
pip install -r waiver_level_extraction_agent/requirements.txt
```

This installs the ADK itself (pinned to version 2.0.0, the version this guide's screenshots come from), the Google cloud libraries, and the PDF-reading libraries. It takes a few minutes.

3.4 Install poppler

System	Command
macOS (Homebrew)	`brew install poppler`
Ubuntu / Debian	`sudo apt install poppler-utils`
Windows	Download a poppler build, unzip it, and add its `bin` folder to your PATH.

3.5 Install the Google Cloud command-line tool and sign in

1. Install the **gcloud CLI** from Google's page: `cloud.google.com/sdk/docs/install`.
2. Sign in so that the tool can use your Google account. This sets up **Application Default Credentials (ADC)**:

   ```
   gcloud auth application-default login
   ```

   A browser window opens. Choose your Google account and allow access. When it finishes, the terminal confirms where the credentials file was saved.
3. Do **not** set the `GOOGLE_APPLICATION_CREDENTIALS` variable. Google's documentation says that variable overrides the credentials you just created.

3.6 Find your Google Cloud project ID, enable billing and turn on Vertex AI

1. Open the Google Cloud console (`console.cloud.google.com`) and sign in.
2. Click the **project picker** at the top of the page and choose your project. The project's **name**, **number** and **ID** are shown on the console's Welcome page, and also on the Home page under "Project info".
3. Copy the **ID**, not the name and not the number. A project ID is made of lowercase letters, numbers and hyphens, for example `my-waiver-project-123456`. It can be chosen when the project is created and cannot be changed afterwards.
4. Make sure **billing** is enabled for this project (see 2.2). If you use an organization's account, an administrator may have to do this for you; that case is not covered here.
5. Turn on the Vertex AI API for the project, either in the console's API library or with:

   ```
   gcloud services enable aiplatform.googleapis.com --project=YOUR_PROJECT_ID
   ```

Google's own documentation has pictures and the current click paths for finding your project ID. The links are in Appendix B.

You can also list your projects from the terminal: `gcloud projects list` prints each project's name, ID and number.

3.7 Create your `.env` file

The `.env` file holds your personal settings. It is never shared or uploaded. Copy the template that comes with the project:

```
cp waiver_level_extraction_agent/.env.example waiver_level_extraction_agent/.env
```

Open `waiver_level_extraction_agent/.env` in any text editor and replace the two highlighted values with **your project ID** from 3.6:

```
PROJECT_ID=your-project-id
GOOGLE_CLOUD_PROJECT=your-project-id
GOOGLE_GENAI_USE_VERTEXAI=TRUE
LOCATION=us-central1
```

> **These two lines must be the same value**
> `PROJECT_ID` is read by the batch scripts and `GOOGLE_CLOUD_PROJECT` is read by the ADK web app. If they differ, or if you leave the placeholder `your-project-id`, the tool stops at start-up and prints a message telling you what to fix. If only one is set, the tool copies it to the other.

Everything else in the file can stay as it is. Appendix A explains each setting. **Never email, upload or commit your `.env` file.**

4. Start the app and open it in your browser

4.1 Start it

In a terminal opened in the project root, with the environment active (the prompt starts with `(.venv)`):

```
adk web . --port 8000
```

Wait until the terminal prints a line like `Uvicorn running on http://127.0.0.1:8000`. **Leave this terminal window open.** The app stops if you close it. The terminal also prints a running log, which is the first place to look if something seems stuck. One server runs all the apps, so if you already started it for the waiver-level guide there is nothing more to start.

4.2 Open the page and choose the services app

Open Google Chrome and go to `http://127.0.0.1:8000`. You see the ADK welcome page (Figure 1). Click **Select an app** and choose **`services_extraction_agent`** (Figure 2).

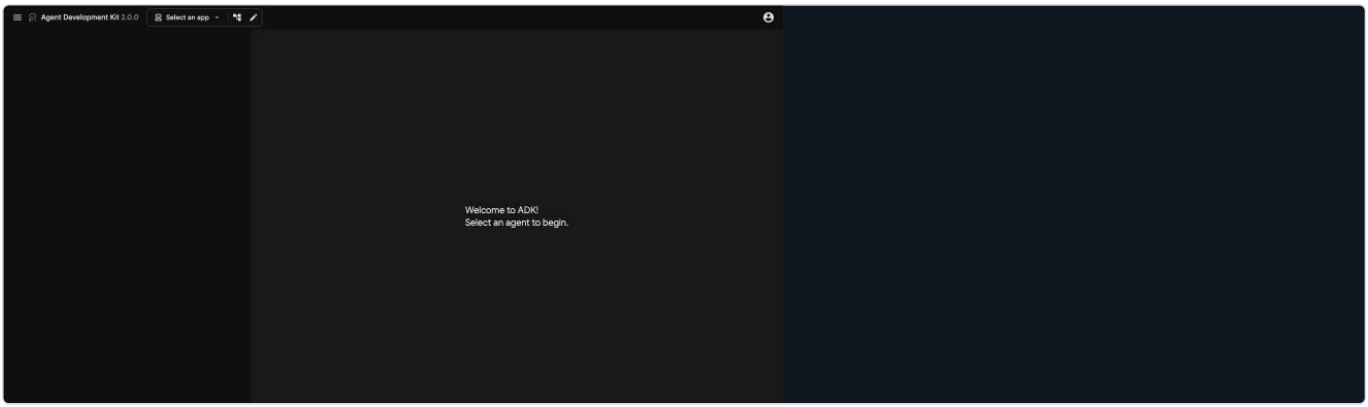


Figure 1. The ADK welcome page, before any app is selected.

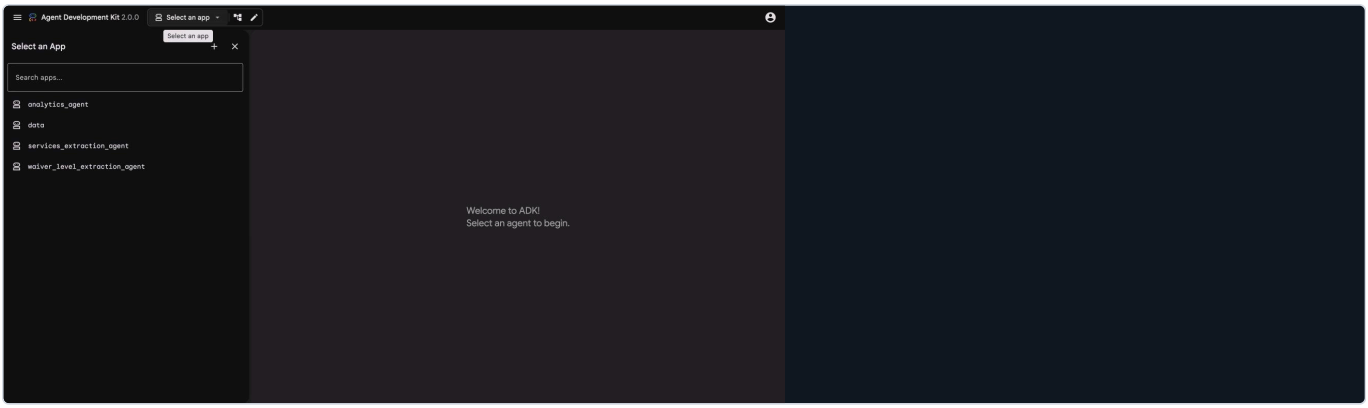


Figure 2. The app list. Choose services_extraction_agent for service-level work. The entry called "data" is just the data folder and can be ignored.

> **Check your Cloud Storage permission before a large run**
> For large PDFs the services extraction caches the PDF in a Google Cloud Storage bucket. If you do not set `CACHE_GCS_BUCKET` in `.env`, a bucket named `<PROJECT_ID>-waiver-pdf-cache` is created in your project the first time it is needed, which requires permission to create buckets. See Appendix A.

4.3 Stop the app

Click in the terminal window and press **Ctrl+C**. Your saved results are not affected.

5. A tour of the screen

The services app uses the same chat window as the waiver app.

Part of the screen	What it does
Message box (bottom)	Type your request and press **Enter**. Everything is plain English.
New Session (top)	Starts a fresh conversation. Click it before starting a different waiver. Results already saved to files are not lost.
Events / Traces (top of the chat)	**Events** shows the conversation and each step the tool takes (the grey tool-call chips). **Traces** shows how long each step took and how many model calls were made, which is the number that drives cost.
Info, State, Artifacts, Evals tabs (left)	**Info** shows details of the event you click. **State** shows the data the session holds in memory; it also shows file paths from your computer, so keep that in mind before sharing a screenshot. **Artifacts** and **Evals** are ADK developer features this tool does not use.

> **Ask the tool what it can do**
> Type `Hi, what can you do?` at any time for a summary.

6. Extract the services in a waiver

6.1 Tell the tool which waiver

You can identify the waiver in several ways:

- By **waiver key**, for example `Extract the services in PA0324R0600`. The PDF is looked up in the default folder (`data/pdfs_267_latest`).
- By **path to a PDF file**, anywhere on your computer.
- By **path to a folder** of PDFs. The tool lists the PDFs in it and processes them one at a time.
- By asking for **the next waiver(s)** without naming one (queue mode, section 6.7).

> **Use a new session for each waiver**
> Click **New Session** before starting a different waiver, so nothing from the previous waiver can be mixed in.

6.2 What happens during a full extraction

The tool hands the job through a series of specialist steps. You can watch them appear as grey chips in the chat (Figure 3).

Step (tool or specialist)	What it does
`get_pdf_path`	Finds and confirms the PDF.
`discover_service_list`	Reads the PDF's form data and works out the list of services. If the waiver was already extracted at waiver level, it uses that list; otherwise the model finds the services in the PDF. The list is cross-checked against the number of service blocks in the form data. It also reuses any services already saved by an earlier run (section 6.5).
`correlate_services_and_taxonomy`	Compares the service list with the form data and the taxonomy information and reports how confident the match is.
`get_waiver_metadata`	Gets the waiver's state, title and file name.
`service_extraction_agent`	Extracts **one service per request**, with its children, in one model call. This repeats for every service in the list, one after another, and is the longest part of a run.
`services_normalisation_agent`	Tidies and standardises the extracted values.
`services_critic_agent`	Checks the results against the form data and the consistency rules, applies corrections it is sure about, and decides: **pass**, **attempt2** (re-read some services) or **human** (needs a person).
`services_reextraction_agent` (only if needed)	Re-reads the flagged services once, then normalisation and the checker run again. This is the last attempt.
`infer_taxonomy` and `finalize_and_save`	Fills in missing taxonomy values and saves `services.json`, the evidence file and the database rows. **It does not write the CSV.**
`review_services`	Builds the review table you check (chapter 7).



Figure 3. The extraction specialist has finished one service (Day Habilitation, with its two children) and control returns to the coordinator.

6.3 Timing and cost

Time and cost depend on how many services the waiver has. There is one extraction call per service, plus the discovery, checking and any re-reading. In our test, extracting **one** service and running it through the rest of the pipeline took about **4 minutes**: roughly 45 seconds to discover the service list, about a minute to extract the service, and the remainder for normalising, checking and saving. A waiver with 15 services is therefore a much longer run.

> **Do not send the request again while it is working**
> A moving progress bar under the chat means the tool is still working. If you resend the request you start a second run and pay for it twice. If nothing has appeared for several minutes, look at the terminal log (see section 12).

6.4 The result summary

When the pipeline finishes, the tool prints a short summary, then the first page of the review table (Figure 4).

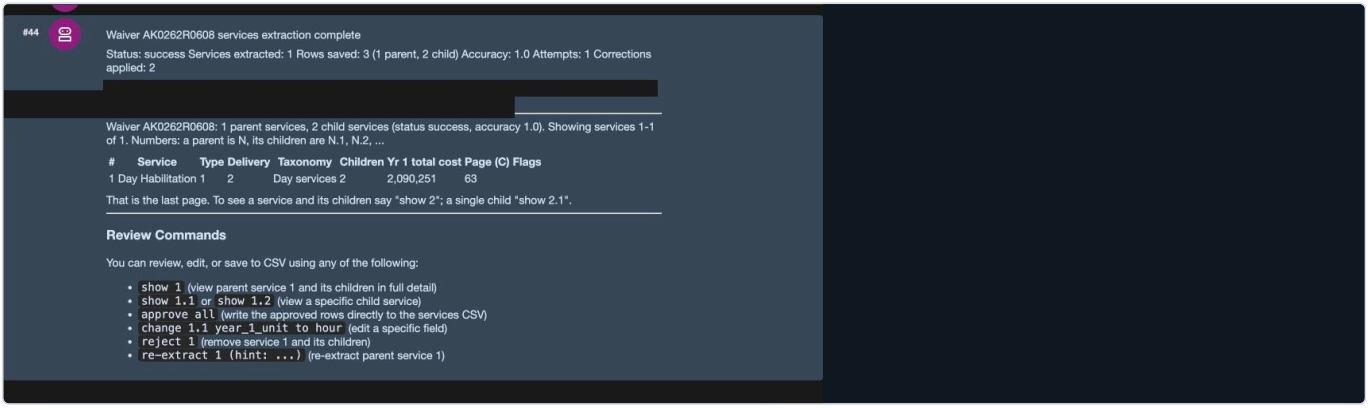


Figure 4. The summary and the first review table. The Output line (the folder where the files were written) is hidden here because it contains a path from the test computer.

Line	Meaning
Status	**success**: the checker passed. **human_review_required**: some flags could not be resolved automatically, so look at them in the review. **validation_failed**: accuracy was below 85% and the result did not pass; check it in the review.

Services extracted	The number of parent services.
Rows saved	Total rows, split into parent and child rows.
Accuracy	The checker's score for this waiver. The evidence and the flags in the review give the detail.
Attempts	1, or 2 if flagged services were re-read.
Corrections applied	How many values the checker corrected from the form data.
Output	The location of the saved `services.json`.

If the status is **human_review_required**, the tool also tells you how many flags and human-queue items remain. Open the flagged services in the review (section 7.4) and decide what to do.

6.5 Resuming an interrupted run

Each service is saved to a checkpoint file as soon as it is extracted. If a run stops part-way, start the app again, open a new session and ask for the same waiver. The tool finds the checkpoint, **reuses every service that was already extracted** and extracts only the rest. It tells you how many services were reused, which is why the run is faster than a full one. Only if you ask for a completely fresh extraction does it ignore the checkpoint.

6.6 Extract or re-extract only named services

You do not have to run the whole waiver. Name the services you want, for example:

- `Extract only the Day Habilitation service from AK0262R0608`
- `Re-extract Respite for PA0354R0507`
- `Fix the self-direction fields on Community Participation Support, it is provider-managed only`

The tool then runs **targeted mode**: it finds the PDF, loads what is already good from the checkpoint (cheap, no extra model calls), and asks the extraction specialist to read just the services you named, one model call each. Whatever you told it ("the cost table is on pages 210 to 215", "this field should be X not Y") is passed along as guidance for that attempt only. A re-extracted service **replaces** the old entry; it is never duplicated.

The tool does **not** save automatically after a targeted extraction. It shows you the result and asks whether to continue through the rest of the pipeline (normalise, check, save), or whether you only wanted to look. Answer `yes, continue and save it` to fold the fix into a proper saved result, after which the review table appears.

6.7 Queue (batch) mode

If you ask for work without naming a waiver, for example `Process the next waiver`, `Do the next 5` or `Continue the batch`, the tool picks the waivers itself:

- It looks at the default PDF folder (`data/pdfs_267_latest`) in alphabetical order.
- It skips waivers that already have services saved in the database.
- The number it takes is the number you asked for, 1 for "the next waiver", or 20 for a bare "continue the batch".

It then runs the full extraction for **one waiver at a time**, finishing each one (and reporting its result) before starting the next. **Your request counts as approval for the whole batch**: it stops only if something needs a decision, such as a waiver that errors out. **Queue mode does not pause for review.** At the end it gives a summary across all the waivers (status, accuracy, services and rows for each, and how many are still pending) and asks whether to run another batch; it

never starts a second batch on its own. You can review any finished waiver later with `Review the services for <KEY>`.

> **Large batches and Google's rate limits**
> If you see quota or 429 errors during a long batch, Google is limiting the request rate. Wait a few minutes, then continue; a pause between waivers helps.

7. Review and correct the results

7.1 How the numbering works

The review is a two-level tree. **A parent service is numbered N. Its children are N.1, N.2, N.3, ...**

- A **parent number** (for example `2`) always means the parent **and all of its children**.
- A **child number** (for example `2.3`) means just that one child.

The numbers are fixed while you review. After you reject something the removed rows are gone, so the tool shows a refreshed table with the new numbers; always use the numbers in the latest table.

7.2 The list of parent services

The first table has one line per parent service, **50 at a time**. Say `show next page` for the next 50, or `show all` to see every parent at once.

Column	Meaning
#	The parent's number.
Service	The service name.
Type, **Delivery**	The service type and delivery method codes (section 7.3).
Taxonomy	The HCBS taxonomy category.
Children	How many child services it has.
Yr 1 total cost	The year 1 total cost.
Page (C)	The PDF page in Appendix C where the service is described.
Flags	How many problems the checker flagged for this service. Open a flagged service to see them.

7.3 What the codes mean

Field	Codes
`service_type`	1 = Statutory Service; 2 = Extended State Plan Service; 3 = Supports for Participant Direction; 4 = Other Service
`service_delivery_method`	1 = Participant-directed as specified in Appendix E; 2 = Provider managed; 3 = Both

`provider_eligibility`	1 = Legally Responsible Person; 2 = Relative; 3 = Legal Guardian. If all apply, 123. An option that does not apply is coded 0, for example 023, 120 or 103.
`provider_specifications`	1 = agency; 2 = individual; 3 = both
`self_direction_authority`	1 = Employer Authority; 2 = Budget Authority; 3 = Both. Blank if the service does not qualify for self-direction.
`information_assistance`	1 = Yes
`year_N_capitation`	1 = Yes, 0 = No
`year_N_component_cost`	Calculated as users times average units per user times average cost per unit. Compare it with the figure in the waiver.

7.4 Opening a service

Type `show 1` to open parent 1 (Figure 5). You see its fields with the evidence for each, the pages where it was found in Appendices C, E and J, a table of costs by year, the checker's flags with their reasons, and a table of its children.

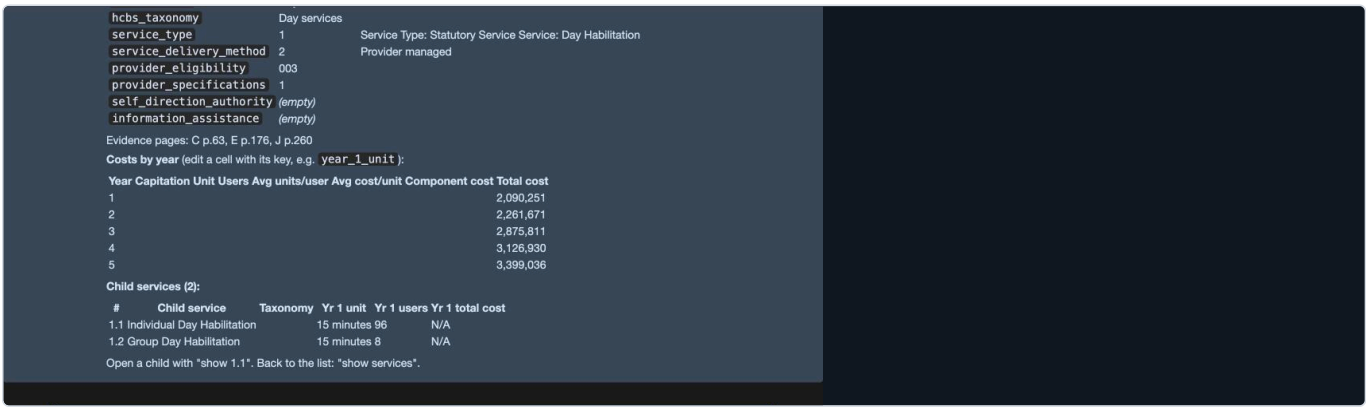


Figure 5. Parent 1, Day Habilitation: its fields and evidence pages, costs by year, and its two children, numbered 1.1 and 1.2.

Type `show 1.1` to open one child. A child shows its name, taxonomy and its own year-by-year units, users and costs. Type `show services` to go back to the list.

7.5 Searching

`search respite` lists every parent or child whose name or taxonomy contains the word, each with its number, so you can open it with `show <number>`. You can search for several words separated by commas.

7.6 Commands you can type

Type this	What happens
`show 2` / `show 2.3`	Opens a parent (with its children) or one child.
`show next page` / `show all` / `show services`	Next 50 parents / every parent at once / back to the list.
`search <words>`	Finds services and children by name or taxonomy.

`change 2.2 year_1_unit to hour`	Changes one field of one row. Use the field names shown in the review (section 1.2). Works on a parent too: `change 3 hcbs_taxonomy to 08010`.
`reject 3.2`	Removes one child.
`reject 3`	Removes parent 3 **and all its children**. The tool tells you so when it does.
`re-extract 2 (hint: ...)`	Re-reads parent 2 from the PDF, with your hint. Its children come with it.
`approve all`	Accepts everything as shown and writes the CSV.
`approve 1-3`, `approve 2.1-2.4`, `approve all except 5`	Accepts part of the table (see below).

Approving part of the table. `approve 1-3` keeps services 1 to 3 with all their children and removes the rest. `approve 2.1-2.4` keeps parent 2's own row with only children 2.1 to 2.4 and removes everything else.

> **Saving happens straight away**
> `approve`, `change` and `reject` each **write the services CSV immediately**. A `change` or `reject` keeps every other service as it was extracted, so after one of these commands the rest of the table has also been saved. Before every overwrite, the previous CSV is copied to `data/csv_backups/`. The tool will not let you remove every service in one step.

7.7 Changing a value

`change 1.2 year_1_unit to hour` edits one field. The tool confirms what it changed and shows a refreshed table (Figure 6). The saved `services.json`, the database and the CSV all receive the change, and the original is kept as `services.pre_review.json`. To undo a change, change the value back.

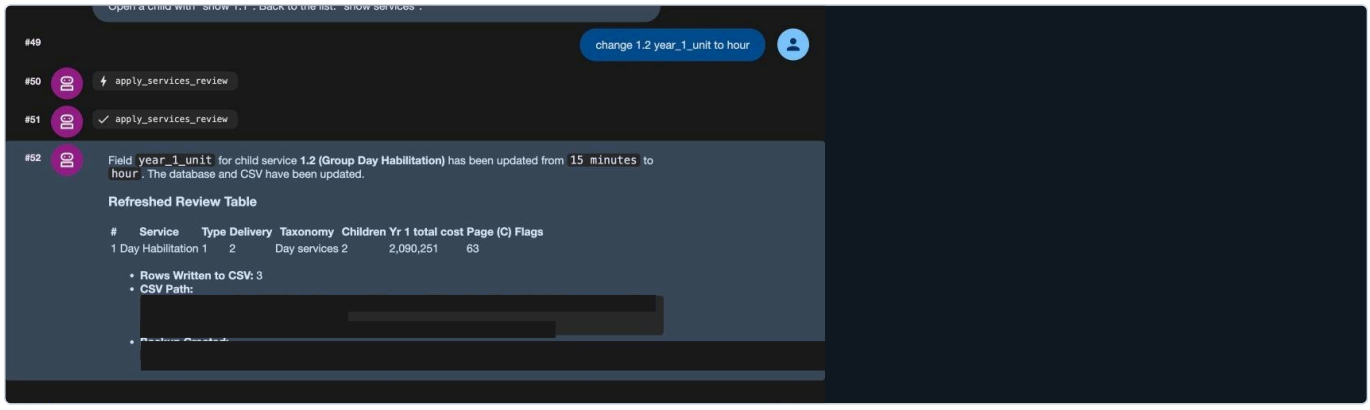


Figure 6. A change applied to child 1.2. File paths in the reply are hidden in this picture.

7.8 Rejecting a service or a child

`reject 3.2` removes child 3.2. `reject 3` removes parent 3 and every child it has, because a child without its parent would be a row with no parent. The removed rows disappear from the saved result and the CSV. The original, with everything, stays in `services.pre_review.json`.

7.9 Re-extracting a service with a hint

When you think the tool missed or misread a service, type `re-extract 2 (hint: the costs are in Appendix J, page 260)`. The tool looks up the exact service name, then runs **targeted mode** (section 6.6) for it: the service is read again with your hint, the result goes through normalisation and the checker, it is saved, and the review for that service is shown again. Re-extraction always works on the **parent**. If you name a child (for example 2.2), the tool tells you that the parent is re-extracted and the children come with it.

7.10 Writing the CSV

`approve all` (or any approve, change or reject) writes this waiver's rows to `data/outputs/service_level_extraction.csv`, or to a different file if you name one. The CSV holds **all waivers' rows**: writing a waiver **replaces only that waiver's rows** and leaves every other waiver untouched.

If you want to write a waiver's result to the CSV **without reviewing it**, say so (for example `write PA0324R0600 to the CSV without review`). You can ask for a preview first, which shows what would change and writes nothing.

8. Extract a value about a service

Sometimes you need something that is not one of the standard columns, for example the **provider qualifications for Day Habilitation**, the **unit of service for one child**, or **which services allow family caregivers**. The custom value feature extracts one such value from the PDF, shows you the evidence, and saves it separately. **It never changes the standard services results.**

8.1 Ask for it

Type, for example, `Extract the provider qualifications for Day Habilitation from AK0262R0608`. The tool first checks whether an existing column already covers it (taxonomy, service type, delivery method, eligibility, specifications, self-direction, units, users or costs). If one does, it shows that and asks whether you want it instead.

8.2 Confirm what you are asking for

If the value is new, the tool shows a **confirmation card** (Figure 7):

- **Service**: which service it is about, a parent or a child, named as in the review table. It can also be left out for a value about the waiver's services in general.
- **Name**, **Meaning** and **Usually contains**: what the value is and what it normally looks like.
- **Type**: text, number, yes/no (1 or 0), date or list. **Where to look**, and **Allowed values** if there is a fixed list.
- The rule that **nothing is guessed**: if the PDF does not say it, the value is left blank.

The tool fills in a suggestion for anything you did not say. If the card is wrong, tell the tool what to change in plain words. You can also say **"you decide"**. When the card is right, type `yes`.

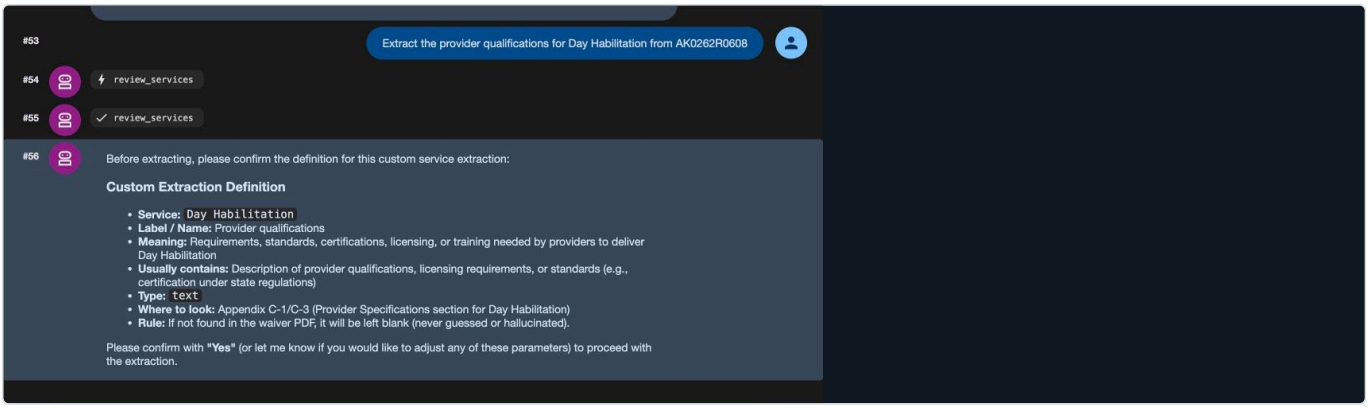


Figure 7. The confirmation card. Nothing is extracted until you answer yes.

8.3 The result

After `yes` the tool makes **one model call** (about 30 seconds in our test). The result shows the value, the page, the exact wording from the PDF, the confidence and any notes, other occurrences of the same item on other pages, and your four choices (Figure 8).

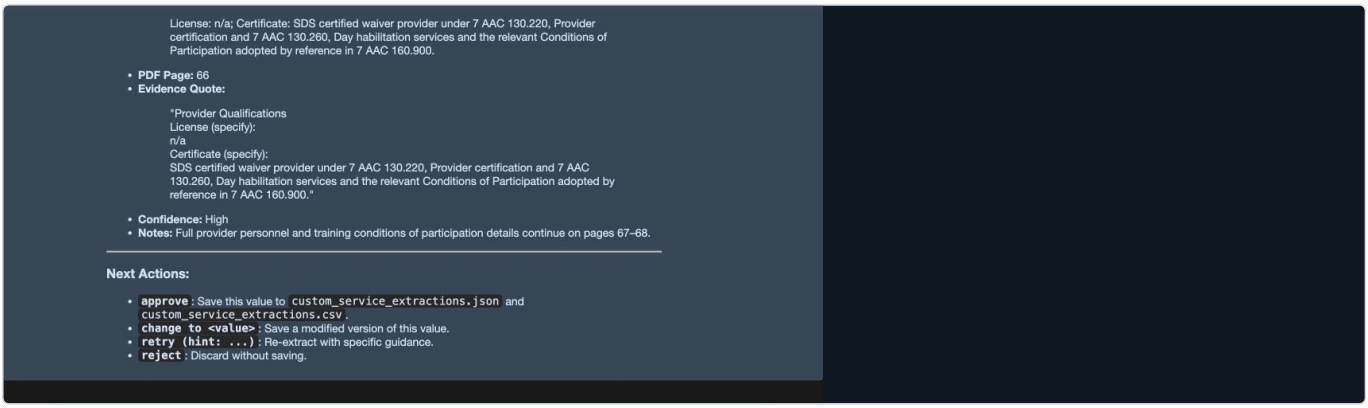


Figure 8. The extracted value, the page (66), the verbatim evidence and the four choices.

> **Always check the evidence**
> In the test, the PDF text on page 66 under "Certified HCBS: Day Habilitation Services" matched the tool's quote exactly. Read the quoted wording and page number every time.

8.4 Your four choices

Type	What happens
`approve`	Saves the value to the waiver's `custom_service_extractions.json` and adds one row to `data/outputs/custom_service_extractions.csv`.
`change to <your value>`	Saves the value you typed instead of the extracted one.
`retry (hint: ...)`	Runs the extraction again with your hint, for example `retry (hint: include the provider category and type too)`. The hint is used for that attempt only and is **not** added to the saved definition.

`reject`	Discards the result. Nothing is saved.

Each row of `custom_service_extractions.csv` has: `waiver_key`, `service_name`, `label`, `value`, `page`, `evidence`, `confidence`, `description`, `value_type`, `model`, `extracted_at` and `approved_at`. Saving the same waiver, service and label again replaces that row.

9. Pipeline mode (command line)

Pipeline mode runs the same extraction without the chat window. You type a command in the terminal and it processes one waiver, or many, one after another. The program is `run_services_pipeline.py`.

> **No review step, and it costs credits**
> Pipeline mode does **not** stop for your approval: each waiver's rows are written to the services CSV as soon as the waiver is finished. **It calls the Gemini model at least once per service and is billed to your Google Cloud project, so a large batch can use a lot of credits.** Start with one waiver, or with `--limit 2`, and check the results and your billing before running a batch. To review a waiver you extracted this way, open it in ADK mode with `Review the services for <KEY>`.

9.1 Before you run a command

Open a terminal in the project root and activate the environment (section 3.2). The same `.env` settings are used. If the project settings are wrong, the command stops at once with a message saying what to fix.

9.2 Where the list of services comes from

For each waiver the program needs the list of its services. It uses, in this order: the list saved when the waiver was extracted at waiver level (`overview.json`), or, if there is none, or if you add `--standalone`, the model finds the services in the PDF. The list is cross-checked against the PDF's form data either way.

9.3 Commands

You must give exactly one of `--waiver`, `--file`, `--all`, `--export-csv` or `--rebuild`.

Command	What it does
`python run_services_pipeline.py --waiver PA0324R0600`	Extracts the services of one waiver by key. The PDF is looked up in the default folder or in `--pdf-dir`.
`python run_services_pipeline.py --file /path/to/waiver.pdf`	Extracts one PDF from any location.
`python run_services_pipeline.py --all`	Extracts every waiver that has an `overview.json` (that is, every waiver already extracted at waiver level).
`--limit 10`	With `--all`: stops after 10 waivers. Use it to test.
`--skip-existing`	With `--all`: skips waivers that already have services saved.
`--pdf-dir ~/Desktop/new_pdfs`	Looks for the PDFs in this folder instead of the default one.

`--output-csv my_run.csv`	Writes the results to this CSV instead of the default `data/outputs/service_level_extraction.csv`. A bare file name goes in `data/outputs/`.
`--standalone`	Finds the service list in the PDF with the model instead of using `overview.json`.
`--waiver KEY --resume`	After an interrupted run, skips the services already saved in its checkpoint and extracts only the rest.
`--waiver KEY --only-service "Respite"`	Extracts just that one named service (one model call) and adds it to the checkpoint, with no checking and no database save. Use it to retry one service that keeps failing. Follow with `--resume` (or `--rebuild`) to fold it into the saved result.
`--rebuild KEY`	Rebuilds the outputs from the latest checkpoint of a crashed run. **No model calls.**
`--export-csv data/outputs/my_export.csv`	Exports every service saved in the database to a CSV. **No extraction and no model calls.**

9.4 Reading a command in plain English

A pipeline command is several small instructions typed on one line. Each part that starts with two dashes (`--`) is an **option** that changes what the program does. Here is one command, taken apart:

```
python run_services_pipeline.py --all --pdf-dir ~/Desktop/new_pdfs --output-csv
my_run.csv
```

Part	What it means
`python run_services_pipeline.py`	"Run the services extraction program."
`--all`	"Do every waiver that has already been extracted at waiver level (every waiver with an `overview.json`), one after another."
`--pdf-dir ~/Desktop/new_pdfs`	"Find the PDFs in this folder." It is the folder called `new_pdfs` on your Desktop; the `~` is shorthand for your own home folder. Without this option the program uses `data/pdfs_267_latest` inside the project.
`--output-csv my_run.csv`	"Save the results in a spreadsheet file called `my_run.csv`" in `data/outputs/` instead of the usual results file.

Put together, it says: *"For every waiver that has already been extracted at waiver level, take its PDF from the new_pdfs folder on my Desktop, extract its services one waiver after another, and save the rows in data/outputs/my_run.csv instead of the usual results file."* The order of the options does not matter.

9.5 Results

Pipeline results go to the same places as in ADK mode (chapter 10). The program writes each waiver's rows into the services CSV as it finishes that waiver, replacing only that waiver's rows, and prints a summary at the end: how many

waivers succeeded, needed human review or failed, the mean accuracy, and the number of services and rows (parent and child).

10. Where your files are saved

All paths are inside the project folder.

File or folder	What it holds
`data/outputs/service_level_extraction.csv`	The approved service-level results. One row per service (parents and children), all waivers together, with 49 sheet columns plus `Row Type`, `Parent Service` and `Waiver Key`. Created (with just the header row) as soon as the app starts.
`data/outputs/custom_service_extractions.csv`	Approved custom service values, one row per waiver, service and label.
`data/outputs/<KEY>/services.json`	The waiver's current result: every parent with its children, and the evidence-based values.
`data/outputs/<KEY>/services.pre_review.json`	The result as it was before your first change or rejection.
`data/outputs/<KEY>/services_evidence.json`	The evidence for each value (pages and wording) and the checker's flags. `services_evidence.pre_review.json` is its copy from before the review.
`data/outputs/<KEY>/services_review.json`	A record of your review decisions.
`data/outputs/<KEY>/services_attempt1.jsonl`	The checkpoint: one line per extracted service, saved as each one finishes. This is what lets an interrupted run resume.
`data/outputs/<KEY>/custom_service_extractions.json`	Custom values for this waiver, with notes and other occurrences.
`data/outputs/waivers.db`	The project database. Its services table holds the saved rows.
`data/csv_backups/`	A copy of the CSV taken before every overwrite.
`data/logs/<KEY>.jsonl`	A step-by-step log of each run for that waiver.
`data/services_human_overrides.jsonl`	An audit record of values the checker corrected from the PDF's form data.

11. Good habits

- **One waiver per session.** Click New Session before starting another waiver.
- **Open the flagged services.** A number in the Flags column means the checker found something that does not add up.
- **Check the evidence, not just the value.** Read the quoted wording and page number, especially for costs and codes.
- **Check parents against their children.** A parent's total should match its children's component costs; the checker flags the ones that do not.
- **Remember that approve, change and reject save immediately.** Do your corrections, then finish with `approve all` so you know the final state is saved.
- **Wait for the progress bar.** Do not resend a request while the tool is still working.
- **Test with one waiver, or one service, before a batch**, and watch your Google Cloud spending (section 2.2).
- **Never share your `.env` file or screenshots that show file paths you do not want shared.**

12. Troubleshooting

What you see	What to do
The terminal says **"Google Cloud project is not set"**, or **PROJECT_ID and GOOGLE_CLOUD_PROJECT differ**, and the app will not load	Open `waiver_level_extraction_agent/.env`. Set both `PROJECT_ID` and `GOOGLE_CLOUD_PROJECT` to your project ID (3.7). Also check that no old value is set in your shell: type `echo $PROJECT_ID $GOOGLE_CLOUD_PROJECT`. Restart `adk web`.
A permission error (403) or "billing" error from Google	Check that billing is enabled for the project and that the Vertex AI API is on (3.6). Re-run `gcloud auth application-default login` and make sure you signed in with an account that has access to that project.
A permission error mentioning **storage** or a **bucket**	Large PDFs are cached in a Cloud Storage bucket. Your account needs permission to create or use one. Create a bucket yourself in the console and put its name in `.env` as `CACHE_GCS_BUCKET` (Appendix A).
A "quota", "resource exhausted" or 429 error	Google is limiting how fast you can make requests. Wait a few minutes and try again; in a batch, leave a gap between waivers.
Nothing appears for a long time after you press Enter	A single service can take about a minute, and the first request after starting the app can take 2 to 3 minutes. Look at the terminal: new lines mean it is still working. Look at the Traces view for the latest step. If there has been no activity for more than 10 minutes, stop the app (Ctrl+C), start it again, and ask for the same waiver; finished services are reused (6.5).
"No services found"	The tool could not find a service list in the PDF. Check that the file is a 1915(c) waiver application and that it is the right PDF. For the command line, try `--standalone`.
Queue mode says nothing is pending	Every PDF in the folder already has services saved. Name a waiver to extract it again.
The status is **human_review_required** or **validation_failed**	The checker could not resolve some flags. Open the flagged services (section 7.4), compare with the PDF, then change, reject or re-extract.

"PDF not found"	Check the waiver key spelling, put the PDF in `data/pdfs_267_latest/`, or give the full path to the file.
The app list contains "data"	It is only a folder. Choose `services_extraction_agent`.
The page will not open at all	Make sure the terminal running `adk web` is still open, and that the address is exactly `http://127.0.0.1:8000`. If port 8000 is in use, start with another port, for example `adk web . --port 8001`.

13. Quick reference

13.1 Setup commands (once)

```
git clone <repository-url> 1915c_extraction_agent && cd 1915c_extraction_agent
python3 -m venv .venv && source .venv/bin/activate
pip install -r waiver_level_extraction_agent/requirements.txt
brew install poppler                        # or: sudo apt install poppler-utils
gcloud auth application-default login
cp waiver_level_extraction_agent/.env.example waiver_level_extraction_agent/.env
# edit .env: set PROJECT_ID and GOOGLE_CLOUD_PROJECT to your project ID
```

13.2 Each time you work

```
cd 1915c_extraction_agent
source .venv/bin/activate
adk web . --port 8000
# open http://127.0.0.1:8000 and choose services_extraction_agent
```

13.3 Chat commands

Type	Purpose
`Extract the services in <WAIVER KEY>`	Run a full services extraction.
`Extract only the <service> service from <KEY>`	Extract or re-extract named services (targeted mode).
`Process the next waiver`, `Do the next 5`, `Continue the batch`	Queue mode (no review stops).
`Review the services for <KEY>`	Open the review table for a waiver already extracted.
`show N`, `show N.k`, `show services`, `show next page`, `show all`, `search <words>`	Move around the review.
`change N.k <field> to <value>`	Correct one field (saves immediately).

`reject N`, `reject N.k`	Remove a parent with its children, or one child (saves immediately).
`re-extract N (hint: ...)`	Re-read a parent service.
`approve all`, `approve 1-3`, `approve 2.1-2.4`, `approve all except 5`	Save the results.
`Extract the <value> for <service> from <KEY>`	Start a custom service value.
`yes`, `approve`, `change to <value>`, `retry (hint: ...)`, `reject`	Answer the custom value prompts.

13.4 Pipeline commands

Command	Purpose
`python run_services_pipeline.py --waiver KEY`	One waiver.
`python run_services_pipeline.py --file /path/to.pdf`	One PDF from anywhere.
`python run_services_pipeline.py --all --limit 2 --skip-existing`	A small test batch of new waivers.
`python run_services_pipeline.py --waiver KEY --resume`	Continue an interrupted waiver.
`python run_services_pipeline.py --rebuild KEY`	Rebuild outputs from a checkpoint (no model calls).
`python run_services_pipeline.py --export-csv data/outputs/my_export.csv`	Export saved services (no model calls).

Appendix A. The `.env` settings

Setting	Meaning
`PROJECT_ID`	Your Google Cloud project ID. **Change this.** Used by the command-line programs.
`GOOGLE_CLOUD_PROJECT`	Your Google Cloud project ID. **Change this.** Used by the ADK web app. Must equal `PROJECT_ID`.
`GOOGLE_GENAI_USE_VERTEXAI`	Leave as `TRUE`. Tells the tool to use Google Cloud's Vertex AI.
`LOCATION`	The Google Cloud region. Leave as `us-central1`. (The Gemini 3.7 model is automatically sent to Google's global endpoint.)

`CACHE_GCS_BUCKET` (optional)	A Cloud Storage bucket used to cache large PDFs during **services** extraction. If it is not set, a bucket named `<PROJECT_ID>-waiver-pdf-cache` is created on first use, which needs permission to create buckets. Set it to the name of a bucket you already have if you cannot create one.
`PDF_DIR`, `OUTPUT_DIR`, `LOG_DIR`	Folders for source PDFs, results and logs, relative to the project root. Leave as they are unless you have a reason to move them.

Appendix B. Google documentation and credits

> **REMINDER: THESE TOOLS NEED CREDITS OR AN ACTIVE BILLING ACCOUNT.**
> **Google charges for the model calls this tool makes, and services extraction makes many per waiver. Enable billing or check your free credits before running, and watch your spend.**

The Google Cloud console screenshots and the exact click paths are in Google's own documentation, which stays up to date with the console's layout:

- **Finding your project ID:** "View and update projects", Resource Manager, docs.cloud.google.com/resource-manager/docs/view-update-projects
- **Creating a project:** "Create projects", docs.cloud.google.com/resource-manager/docs/creating-managing-projects
- **Signing in for local use (ADC):** "Set up ADC for a local development environment", docs.cloud.google.com/docs/authentication/set-up-adc-local-dev-environment
- **Project, billing and API setup for Vertex AI:** "Set up a project and a development environment", docs.cloud.google.com/vertex-ai/docs/start/cloud-environment
- **Billing questions for Vertex AI:** docs.cloud.google.com/vertex-ai/docs/support/billing-questions
- **Free trial credits:** "Google Cloud Free Trial FAQs", cloud.google.com/signup-faqs
- **Current model prices:** cloud.google.com/vertex-ai/pricing
- **Installing the gcloud CLI:** cloud.google.com/sdk/docs/install

Google, Google Cloud, Vertex AI and Gemini are trademarks of Google LLC. Page layouts and product names in Google's documentation may change; use the current version of those pages.

Appendix C. Glossary

Term	Meaning
1915(c) waiver	A Medicaid program that pays for home and community-based services as an alternative to institutional care.
Waiver key	The identifier of one waiver application, such as `PA0324R0600`. Not the same as a Google Cloud project ID.
Parent service / child service	A service as the waiver names it, and a variant of it with its own users and costs. Numbered N and N.1, N.2, ... in the review.
HCBS taxonomy	The standard category scheme for home and community-based services.

AcroForm	The fillable-form data embedded in many PDFs. The tool compares what the model read with it.
Checker (critic)	The step that verifies the extraction, applies safe corrections and flags what does not add up.
Flag	A problem the checker found, for example a parent total that does not match its children's costs.
Checkpoint	The file where each extracted service is saved as it finishes, so an interrupted run can resume.
Queue mode	Asking for "the next waiver(s)" so the tool picks which waivers to process.
Targeted mode	Extracting or re-extracting only the services you name.
Project ID	The unique ID of your Google Cloud project, for example `my-waiver-project-123456`.
ADC (Application Default Credentials)	The way Google's tools find your sign-in on your computer, set up with `gcloud auth application-default login`.
Vertex AI	Google Cloud's service for running AI models, including Gemini.
ADK (Agent Development Kit)	The framework that provides the chat window and runs the tool's steps.
Evidence	The exact wording from the PDF, with its page number, that an answer is based on.
Session	One conversation in the chat window. Start a new one with New Session.

Appendix D. Tested with

Operating system	macOS (**not tested on Windows**)
Python	3.12
Google ADK	2.0.0
Model	Gemini 3.7 Flash (through Vertex AI)
Browser	Google Chrome
Waivers used in screenshots	AK0262R0608 (Alaska), service Day Habilitation; the custom value was read from that waiver's PDF

Windows and Linux commands are given for convenience but were not run for this guide. The pipeline-mode options were documented from the program's help text.

Appendix E. Setting up with an AI assistant

If you already use an AI assistant, it can take the project from a fresh download to a working app on your computer. A **coding assistant that can run commands** (one that works in a terminal or an editor, with your permission) can do almost

all of section 3 itself: download the project, create the Python environment, install the packages and tools, create the `.env` file, and start the app.

E.1 Who does what

The assistant can do	Only you can do (the Google and Gemini side)
Download the project (`git clone`) Check Python, create `.venv` Install the packages, poppler and the gcloud CLI Create `.env` and fill in your project ID Check the settings and start `adk web`	Create or choose the Google Cloud project Turn on billing, or confirm you have credits Turn on the Vertex AI API (or ask the assistant to run the command) Complete the `gcloud auth application-default login` sign-in in the browser Copy the project ID and give it to the assistant

A good assistant stops at each of those steps on the right, tells you exactly what to do, and waits. **It should never start an extraction**, because that calls the Gemini model and is billed to your project.

E.2 The prompt

Open the project folder in your assistant (or an empty folder, if it will download the project for you), then paste the text below. Replace the text in **[square brackets]** first. The same prompt is in the SETUP file in the project folder.

```
You are my setup assistant. Your job: take this project from a fresh download to a
working app on my computer, doing everything you can yourself, and stopping to hand me
the few steps that only I can do.

THE PROJECT
A tool that extracts data from 1915(c) Medicaid waiver PDFs using Google's Gemini
models through Google Cloud (Vertex AI). Repository address: [repository URL, or
"already cloned in this folder"]

ABOUT ME
- Operating system: [macOS / Linux / Windows]
- Google Cloud project ID: [your project ID, or "I don't have one yet"]
- I am using my own personal Google account and my own Google Cloud project.

WHAT YOU DO YOURSELF (explain each command first, and ask before anything that
installs software or changes system settings)
1. Get the project: git clone the repository (skip if it is already here), then work
from the project root folder.
2. Check that Python 3.10 or newer is installed. Create and activate a virtual
environment named .venv in the project root.
3. Install the packages: pip install -r waiver_level_extraction_agent/requirements.txt
4. Install poppler for my operating system (macOS: brew install poppler;
Debian/Ubuntu: sudo apt install poppler-utils).
5. Install the gcloud CLI if it is missing
(https://cloud.google.com/sdk/docs/install).
6. Copy waiver_level_extraction_agent/.env.example to
waiver_level_extraction_agent/.env. When I give you my project ID, set BOTH PROJECT_ID
and GOOGLE_CLOUD_PROJECT to it. They must be identical. Leave the other settings as
```

```
they are.

WHAT ONLY I CAN DO (stop, tell me exactly what to click or type, and wait until I say
it is done)
A. Create a Google Cloud project, or choose an existing one, and give you its project
ID (the ID, not the name or number).
B. Turn on billing, or confirm I have free credits, for that project. Model calls are
charged to it.
C. Turn on the Vertex AI API for the project (console API library, or: gcloud services
enable aiplatform.googleapis.com --project=<ID>). You may run the command if I ask you
to.
D. Sign in with: gcloud auth application-default login (a browser window opens and I
complete it myself).

THEN CHECK THAT IT WORKS (report each result)
1. The project settings are consistent. Run this from the project root with the
environment active:
   python3 -c "from waiver_level_extraction_agent.shared_libraries.config_check import
check_cloud_config as c; c(); print('config ok')"
2. My Google sign-in works (this must not print any token):
   gcloud auth application-default print-access-token > /dev/null && echo "sign-in ok"
3. Start the app: adk web . --port 8000
   Confirm that http://127.0.0.1:8000 loads and that waiver_level_extraction_agent
appears in the app list. (The entry called "data" is not an app.) Leave the app
running and tell me to open that address in my browser.

RULES
- Do NOT run an extraction, or anything that calls the Gemini model, unless I tell you
to. Those calls cost money.
- Never ask me to paste passwords, tokens or key files into this chat. Never write
secrets into files. Never commit, upload or print my .env file.
- Do not enable billing or APIs on your own. Do not change anything outside this
project folder except installing the tools above.
- If a step fails, show me the exact error, explain it in plain language, and suggest
the smallest fix. Common ones: PROJECT_ID and GOOGLE_CLOUD_PROJECT differ (set both to
the same value); a 403 or billing error (billing or the Vertex AI API is not enabled,
or I signed in with the wrong account); poppler not found (install it and reopen the
terminal).
- This project has not been tested on Windows. If something looks Windows-specific,
tell me it may not work instead of guessing.
- If you cannot run commands, give me the commands one at a time, wait for me to paste
the output, and continue from there.
- When you finish, summarize what you installed and changed, and what is still left
for me to do.
```

E.3 What a successful setup looks like

- The assistant reports `config ok` and `sign-in ok`.
- The terminal shows `Uvicorn running on http://127.0.0.1:8000`.
- When you open `http://127.0.0.1:8000` and click **Select an app**, `waiver_level_extraction_agent` is in the list (Figure 2).
- No extraction has been run and no money has been spent yet.

> **Read what the assistant does**
> An assistant can make mistakes or misread an error. Read each command before you approve it. Do the Google sign-in and billing steps yourself. Do not paste passwords, keys or tokens into a chat, and do not let it upload your `.env` file anywhere.

If your assistant cannot run commands (a plain chat window), the prompt tells it to give you the commands one at a time and wait for your pasted output, so you can still follow along.